

FCFM



Universidad Autónoma de Nuevo león

Facultad De Ciencias Físico Matemático

Materia: Señales de transmisión

Doctor: Guillermo Ezequiel Sánchez Guerrero

Diego López Moreno 2142590

Fernando de Jesús Gutiérrez López 2167284

Diego alexander salinas Pérez 2160247

Servando Gael López Murillo. 2159422

PIA - H3: Mensaje transmitido y caracterización del canal

Mensaje transmitido y caracterización del canal

Se envía el mensaje: *Somos el equipo 3 y este es nuestro mensaje*

1. Mensaje de texto recibido.

El mensaje transmitido tiene una longitud de 43 caracteres, Cumple con el mínimo de 32 caracteres solicitados. El mensaje se recibió correctamente de principio a fin en todas las pruebas realizadas.

2. Modulación

Usamos modulación OOK (On-Off Keying). En nuestro código QR, las zonas claras representan el 1 y las zonas oscuras el 0. El patrón de bits forma los caracteres en nuestro mensaje. También usamos el preámbulo: 'UUUU|'. Es como una señal de inicio que le avisa a la cámara que ya comenzara a partir de ahí el mensaje, para que se sincronice antes de leer los datos reales.

2.1 Trama con preámbulo de sincronización

Tasa de bits Datos: Mensaje: 43 caracteres

Preámbulo: 5 caracteres (UUUU|)

Cámara del celular: 30 cuadros por segundo (30 fps)

1 carácter = 8 bits

Cálculos:

Bits del mensaje: 43 caracteres x 8 bits = 344 bits

Bits del preámbulo: 5 caracteres x 8 bits = 40 bits

Total de bits a transmitir = 384 bits

Tasa de bits: la cámara captura 30 imágenes por segundo (30 bps). 30 fps = 30 bits por segundo en nuestro sistema.

Tiempo aproximado:

Cada cuadro/imagen del código QR dura 1/30 de segundo (debido a los 30 fps de la cámara).

Nota**:** Al utilizar un código QR estático, la trama completa de 384 bits se transmite en una sola imagen. A corta distancia (20-60 cm), la decodificación es prácticamente instantánea (aprox 1 seg) debido al enfoque rápido de la cámara. A mayor distancia (100cm), la lectura toma entre 2 y 3 segundos

Límite de NYQUIST:

Para leer bien una señal, necesitas verla al menos el doble de rápido que los cambios que quieres detectar.

Formula: Frecuencia máxima = Frecuencia de la cámara / 2

Nuestros datos: 30 fps / 2 = 15 Hz (lo que nos dice el limite de NYQUIST es que con nuestra cámara a 30 fps, lo mas rápido que podemos cambiar el código QR sin perder información es 15 veces por segundo (15 Hz). Si intentamos hacerlo mas rápido la cámara no alcanza a leer bien y llega con errores).

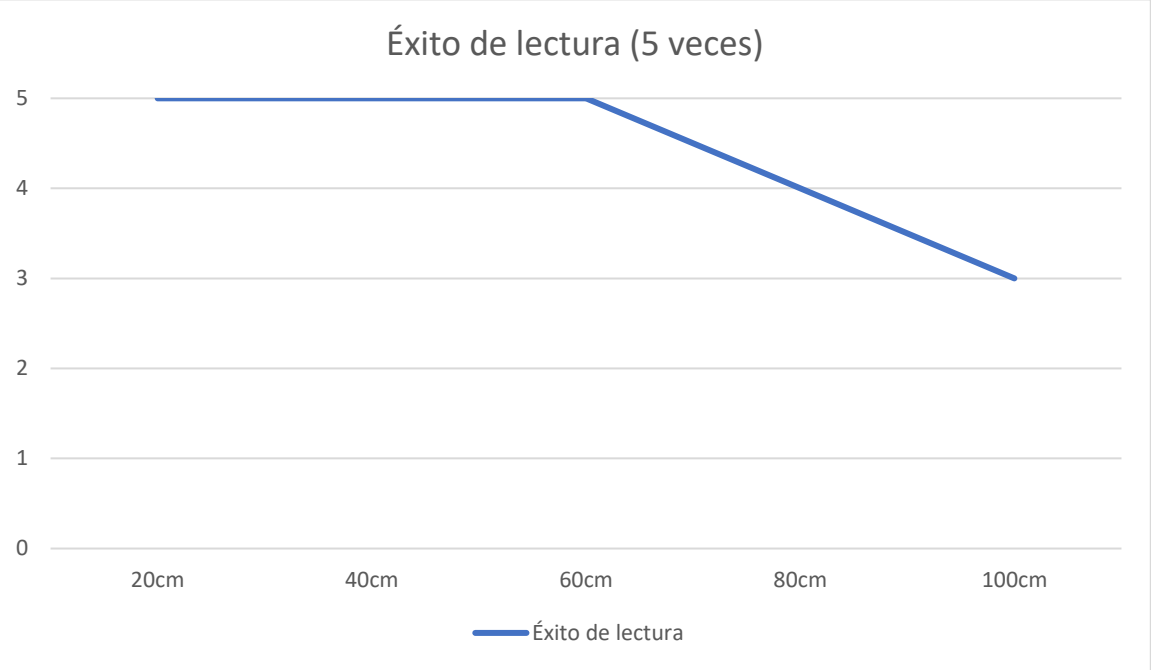
Parámetro	**Valor**
Modulación	OOK(ON-OFF KEYING)
Tasa de bits alcanzada Limite de Nyquist	30 bits por segundo 15 HZ (30 FPS/2)

4. Caracterización del canal

Resultados obtenidos al variar la distancia entre la pantalla y cámara

DISTANCIA (cm)	**¿Se detecto el QR?**	**Mensaje Recibido**	**Observaciones del Canal**
20 cm	*Si 100%*	83111109111115	Lectura inmediata y estable
40 cm	Si 100%	83111109111115	Buena recepción a corta distancia
60cm	SI 100%	83111109111115	Debes de mantener la cámara firme
80cm	Si/no 80%	83111109111115	Ligera perdida de contraste por la distancia
100cm	Si/no 60%	83111109111115	La cámara no distingue bien los módulos, depende mucho de la cámara del o factores que alteren la resolución del QR

SI/NO= LECTURA INTERMITENTE

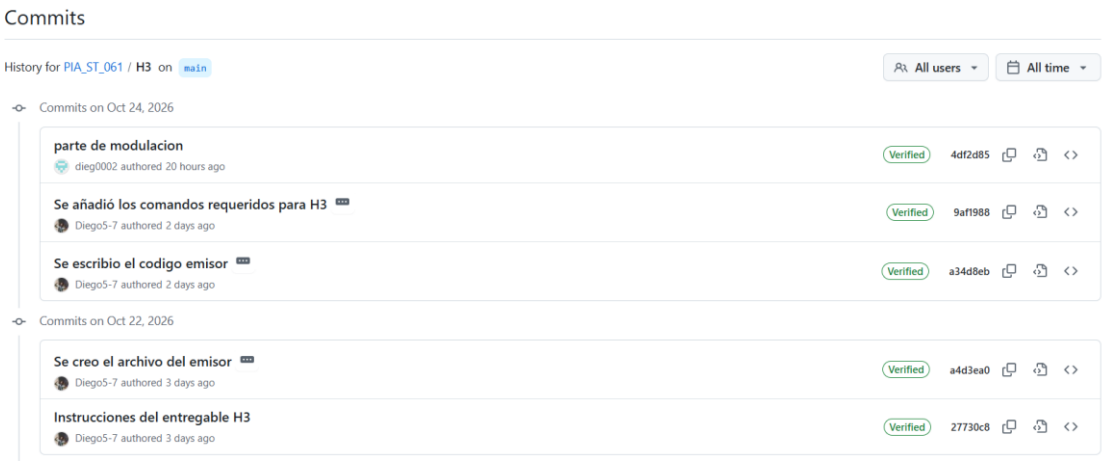


5. Bitácora del equipo H1,H2,H3

Actividad	Resultado	Fecha	Responsable
Cálculo de bits de la trama y de la tasa de bits	384 bits en total;30 bits por segundo	23/10/2026	Diego Alexander Salinas Pérez
Cálculo del límite de Nyquist	15 Hz con cámara de 30 FPS	23/10/2026	Todos
Pruebas de caracterización del canal a 20,40,60,80 y 100 cm	Mensaje recibido en todas las distancias; lecturas menos estables de 80 y 100 c,	24/10/2026	Todos
Generación de los códigos (emisor.py y receptor.py)	Emisor: Trama UUUU| + mensaje de 43 caracteres; imagen código QR. Receptor: Captura el código QR mediante la cámara, valida el preámbulo de sincronización, procesa los valores numéricos del formato ASCII y reconstruye e imprime el	22/10/2026	Diego López Moreno

	mensaje original del equipo.		
Grabación del video de evidencia	Video subido a carpeta de drive (sección 6)	25/10/2026	Servando Gael López Murillo
Redacción del reporte H3	Secciones 1 a 7 integradas	25/10/2026	Fernando de Jesús Gutiérrez Lopez

Los roles asignados Desde H1-H3 son los mismos.



6. Evidencia en video como parte del experimento.

https://drive.google.com/drive/folders/1yTGJGHSKvJxf_uxAa1QT4m0F-eH8o0G7?usp=sharing

7. Código

Código del emisor

emisor.py 1 ✕ $ informacion_web.sh

C: > Users > lenov > Downloads > emisor.py > ...

```python
import qrcode

mensaje_inicial = "Somos el equipo 3 y este es nuestro mensaje"

print(f"Mensaje: {mensaje_inicial}")
print(f"Longitud: {len(mensaje_inicial)} caracteres")

preambulo = "UUUU|"

forma_ascii = " ".join(str(ord(c)) for c in mensaje_inicial)

trama_completa = preambulo + forma_ascii

print(f"Trama con preámbulo: {trama_completa}")

img = qrcode.make(trama_completa)
img.save("codigo_qr_h3.png")
img.show()
```

Código del receptor

C: > Users > diego > Downloads > receptor.py

```python
import cv2

def main():
    opcion = input("Elige el tipo de cámara (0 para laptop, 1 para celular): ").strip()

    if opcion == "1":
        fuente = 1
        print("Cámara de celular seleccionada.")
    else:
        fuente = 0
        print("Cámara de la laptop seleccionada por defecto.")

    cap = cv2.VideoCapture(fuente)
    lector = cv2.QRCodeDetector()

    if not cap.isOpened():
        print("No se pudo abrir la cámara.")
        return

    print("Con la letra 'q' se cierra la ventana.\n")

    ultimo = ""
    preambulo_esperado = "UUUU|"

    while True:
        ret, frame = cap.read()
        if not ret:
            print("No se pudo leer el video de la cámara.")
            break

        data, bbox, _ = lector.detectAndDecode(frame)

        if data and data != ultimo:
            print(f"Mensaje: {data}")
```



```python
def main():

            if data.startswith(preambulo_esperado):
                print("Preámbulo 'UUUU|' sincronizado correctamente.")

                cuerpo_ascii = data[len(preambulo_esperado):]

                try:
                    mensaje_reconstruido = "".join(chr(int(codigo)) for codigo in cuerpo_ascii.split())
                    print(f"Mensaje decodificado: {mensaje_reconstruido}")
                except Exception as e:
                    print(f"Error al decodificar los caracteres ASCII: {e}")
            else:
                print("Error de sincronización: Preámbulo no reconocido.")

            ultimo = data

        cv2.imshow("Receptor OWC - FCFM (H3)", frame)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

8.Enlace al GitHub

https://github.com/Diego5-7/PIA_ST_061